RAG-Driven Customer Support Assistant System.
Deliverable 2- LLD

ADVANCED_GENAI_INTERNSHIP – FEBRUARY 2026

NAMRATA DIGHE - IN226091702

Objective

The objective of this Low-Level Design (LLD) is to define the **internal working and implementation details** of the RAG-based system, including modules, data structures, workflows, and error handling mechanisms.

1. Module-Level Design

The system is divided into multiple functional modules:

- Document Processing Module**:** Loads PDF files and extracts raw text using document loaders.
- Chunking Module**:** Splits text into smaller chunks using recursive splitting (chunk_size = 1000, overlap = 100) to preserve context.
- Embedding Module**:** Converts text chunks into vector embeddings using embedding models.
- Vector Storage Module: Stores embeddings in ChromaDB for efficient similarity search.
- Retrieval Module**:** Retrieves Top-K relevant chunks based on query similarity.
- Query Processing Module: Handles user input and prepares it for retrieval and LLM processing.
- Graph Execution Module (LangGraph): Controls workflow, manages state, and handles routing decisions.
- HITL Module**:** Provides human intervention for low-confidence or complex queries.

---

2. Data Structures

- Document**:** Stores extracted text and metadata.
- Chunk**:** Smaller text segments with source reference.
- Embedding**:** Numerical vector representing semantic meaning.
- Query-Response Schema**:** Contains query, generated answer, and source documents.
- State Object (AgentState**):** Tracks workflow data such as query, retrieved documents, generated answer, confidence score, and HITL flag.

3. Workflow Design (LangGraph)

The workflow is implemented using a graph-based approach:

- **Processing Node:** Retrieves context and generates response using LLM.
- **Output Node:** Returns final answer or triggers escalation.

Flow:
User Query → Processing Node → Decision →
→ Output Node (Auto Response)
→ HITL Node (Escalation)

State Flow:
Query → Retrieved Docs → Answer → Confidence → Decision

---

4. Conditional Routing Logic

The system uses conditions to decide response flow:

- **Answer Generation:**
  - confidence_score ≥ 0.8
  - Relevant documents available
- **Escalation Conditions:**
  - confidence_score < 0.8
  - No relevant chunks found
  - Query is complex or unclear

This ensures the system avoids incorrect or hallucinated responses.

---

5. HITL (Human-in-the-Loop) Design

- Trigger**:** Low confidence, missing data, or complex queries
- Process**:** System pauses and sends query to human expert
- Integration**:** Human response is stored and returned as final output

---

6. API / Interface Design

- **Input Format:**

```
{
  "query": "string",
  "user_id": "string"
}
```

- **Output Format:**

```
{
  "answer": "string",
  "status": "COMPLETED | PENDING_HUMAN"
}
```

- **Interaction Flow:**
  User → API → LangGraph → Retriever → LLM → Response

---

7. Error Handling

- **Missing Input:** Return validation error
- **No Relevant Chunks:** Skip LLM and trigger HITL
- **LLM Failure:** Retry (up to 3 times), else escalate
- **Hallucination Prevention:** Validate output using retrieved context

---

Conclusion

The LLD provides a detailed internal blueprint of the RAG system, defining modules, workflows, data structures, and decision logic. It ensures **accurate, reliable, and scalable document-based question answering**.